SHACL для начинающих: проверка данных по формам

Конспект к лекции 3. Все примеры выполнены pySHACL 0.28 на двух графах: маленькой учебной библиотеке и онтологии примера редактора `data/review.owl`. Отчёты приведены в том виде, в каком их выдала программа. Материал идёт от простого к сложному: сначала одна форма с одним ограничением, в конце запросы и граница с OWL.

1. Зачем нужна проверка данных

Онтология и граф знаний записывают, что есть в предметной области: классы, свойства, индивиды и связи между ними. Машина вывода отвечает на вопрос «что следует из записанного». На другой вопрос она не отвечает: всё ли записано и правильно ли. Если у рецензии не указана статья, машина вывода в OWL скажет: статья есть, она пока не названа. Это открытый мир, и для вывода он правилен. Для приёма данных он бесполезен: редактору нужно знать, что у рецензии действительно записана статья, оценка выставлена по известной шкале, а дата сдачи не раньше даты назначения.

SHACL (Shapes Constraint Language, язык ограничений в виде форм) решает именно эту задачу. Он описывает, как должны выглядеть данные, и выдаёт отчёт о каждом месте, где данные выглядят иначе. SHACL работает в замкнутом мире: чего нет в графе, того нет.

Одна фраза, которую стоит запомнить: машина вывода отвечает, что следует из записанного, форма отвечает, всё ли записано и верно ли.

2. Три участника: данные, формы, отчёт

В проверке участвуют три графа RDF.

Граф	Что в нём	Кто пишет
граф данных	индивиды и их связи: книги, авторы, рецензии	данные области
граф форм	формы: описания того, как должны выглядеть узлы данных	автор проверки
отчёт	список найденных несоответствий, по одному результату на нарушение	программа проверки

Программа проверки (валидатор) берёт первые два графа и строит третий. В курсе используется pySHACL, библиотека Python. Запуск из командной строки и из программы:

```
pyshacl -s shapes.ttl data.ttl
```

```
from pyshacl import validate
ok, report_graph, report_text = validate(data, shacl_graph=shapes, inference="none")
```

Первое возвращаемое значение `ok` отвечает на вопрос «соответствуют ли данные формам», второе есть отчёт как граф, третье тот же отчёт текстом.

3. Первая форма

Учебный граф данных: четыре книги и три автора. Записан в Turtle, префикс `:` обозначает пространство имён библиотеки.

```
:к1 a :Книга ; :название "Логика" ;    :автор :Тарский ; :год 1936 .
:к2 a :Книга ; :название "Онтологии" ; :автор :Лами ; :автор :Груббер ; :год 2021 .
:к3 a :Книга ;                          :автор :Поппер ; :год "1972"^^xsd:string .
:к4 a :Книга ; :название "Знание" ;                      :год 1945 .
:Тарский a :Автор ; :имя "Альфред Тарский" .
:Лами a :Автор .   :Поппер a :Автор .
```

В данных четыре подвоха: у книги к3 нет названия, у книги к4 нет автора, год книги к3 записан строкой, а Груббер нигде не объявлен автором. Формы будут находить их по одному.

Первое требование: у каждой книги есть название. Форма:

```
shp:Книга a sh:NodeShape ;
    sh:targetClass :Книга ;
    sh:property [
        sh:path :название ;
        sh:minCount 1 ;
        sh:message "у книги нет названия" ] .
```

Что делает форма. Берёт каждый узел с типом Книга, смотрит по свойству `название` и считает значения. Если их нет, добавляет в отчёт строку с этой книгой и сообщением. Книги с одним или несколькими названиями проходят.

Как это читается. `shp:Книга` есть узловая форма: описание того, как должен выглядеть узел. `sh:targetClass :Книга` называет цель: проверке подлежат все узлы с типом Книга. `sh:property [...]` вводит одно ограничение на свойство: по пути `:название` должно быть не меньше одного значения. `sh:message` задаёт текст, который попадёт в отчёт.

Отчёт:

```
Conforms: False
  к3   название   у книги нет названия   Violation   MinCountConstraintComponent
```

Каждая строка отчёта содержит пять вещей: узел, в котором найдено несоответствие (к3), путь (название), сообщение формы, строгость (Violation, нарушение) и компонент, то есть какое ограничение сработало (MinCount). Этого достаточно, чтобы найти место в данных и понять, что исправить.

4. Число связей

Два ограничения задают, сколько значений может быть у свойства: `sh:minCount` (не меньше) и `sh:maxCount` (не больше). Вместе они записывают «ровно одно»: minCount 1 и maxCount 1.

Требование «у книги один автор» с мягкой строгостью:

```
sh:property [ sh:path :автор ; sh:maxCount 1 ;
              sh:severity sh:Warning ; sh:message "больше одного автора" ] .
```

Что делает форма. Для каждой книги считает значения свойства `автор`. Больше одного, в отчёт попадает строка со строгостью «предупреждение». Книги с одним автором и книги без автора проходят: форма задаёт только верхнюю границу.

```
  к2   автор   больше одного автора   Warning   MaxCountConstraintComponent
```

У книги к2 два автора, форма сообщила об этом предупреждением. О строгости в разделе 7.

5. Тип значения

Три ограничения говорят, каким должно быть значение.

`sh:class`: значение есть индивид указанного класса. «Автор книги должен быть записан и быть автором»:

```
sh:property [ sh:path :автор ; sh:class :Автор ; sh:minCount 1 ;
              sh:message "автор должен быть записан и быть автором" ] .
```

Что делает форма. Две проверки в одном ограничении. Первая: у книги есть хотя бы одно значение `автор`, иначе нарушение MinCount. Вторая: у каждого значения в графе записана тройка «значение а :Автор», иначе нарушение Class. Отчёт называет, какая из двух сработала.

```
к2  автор  автор должен быть записан и быть автором  Violation  ClassConstraintComponent
к4  автор  автор должен быть записан и быть автором  Violation  MinCountConstraintComponent
```

У книги к4 автора нет вовсе, сработал MinCount. У книги к2 один из авторов, Груббер, не объявлен автором: в графе нет тройки «Груббер а Автор», сработал Class. Это замкнутый мир в действии: форма не рассуждает о том, кем Груббер мог бы быть, она смотрит, что записано.

`sh:datatype`: значение есть литерал указанного типа. «Год есть целое число»:

```
sh:property [ sh:path :год ; sh:datatype xsd:integer ; sh:minInclusive 1450 ;
              sh:message "год: целое число не раньше 1450" ] .
```

Что делает форма. Каждое значение свойства `год` должно быть литералом с типом `xsd:integer` и числом не меньше 1450. Строка «1972» не целое число и не сравнивается с 1450, поэтому даёт два нарушения. Книга без года проходит: о наличии года форма ничего не говорит.

```
к3  год  год: целое число не раньше 1450  Violation  DatatypeConstraintComponent
к3  год  год: целое число не раньше 1450  Violation  MinInclusiveConstraintComponent
```

Год книги к3 записан строкой «1972». Сработали оба ограничения: тип не тот, и сравнение с числом 1450 для строки не проходит. Одно поле, два результата: отчёт перечисляет каждое сработавшее ограничение отдельно.

`sh:nodeKind`: значение есть IRI (узел с именем), литерал или пустой узел. Пример: `sh:nodeKind sh:IRI` для свойства автор проходит на всех четырёх книгах, потому что все авторы записаны именами.

6. Допустимые значения

Четыре ограничения на содержание литералов и узлов.

Ограничение	Смысл	Пример
`sh:in (5 10)`	значение из списка	шкала оценки 5 или 10
`sh:hasValue true`	среди значений есть указанное	ответ авторов записан
`sh:minInclusive 1`, `sh:maxInclusive 10`	числовой диапазон, есть и строгие minExclusive, maxExclusive	оценка от 1 до 10
`sh:pattern "^[А-ЯЁ]"`	регулярное выражение для строк	имя с заглавной буквы

Проверка «имя автора с заглавной» на учебном графе проходит: единственное записанное имя, «Альфред Тарский», ей удовлетворяет. У авторов без имени проверять нечего, и ограничение на содержание их не затрагивает. Чтобы потребовать само наличие имени, нужен minCount.

7. Строгость, сообщения и итог отчёта

У каждой формы и каждого ограничения на свойство может быть строгость `sh:severity`: `sh:Violation` (по умолчанию), `sh:Warning`, `sh:Info`. Строгость попадает в отчёт как пометка и ни на что больше не влияет: валидатор не останавливается и не отбрасывает данные.

Важная деталь: итог `Conforms` равен True только когда отчёт пуст. Предупреждение тоже результат, поэтому данные с одним предупреждением получают `Conforms: False`. Если нужно различать «есть нарушения» и «есть только предупреждения», это делается по полю строгости в отчёте, как в блокноте лекции: «нарушений: 4, предупреждений: 3».

Сообщение `sh:message` пишется так, чтобы по нему можно было найти пункт требования: «6.1: сданных рецензий меньше двух». Без сообщения в отчёт попадёт текст валидатора вроде «Less than 1 values», понятный только автору формы.

8. Пример курса: журнал рецензий

Дальше примеры идут на онтологии примера редактора `data/review.owl`. В ней статьи P1–P8, четырнадцать рецензий вида `рец_P5_Rv3`, рецензенты Rv1–Rv6 с организациями, положение из разделов, терминов и требований. У рецензии записаны свойства: `на_статью`, `рецензент`, `назначено` и `сдано` (даты), `оценка` (десятичное число), `шкала` (5 или 10), `ответ_авторов` (да или нет).

Три проверки записи, которые должны проходить на чистых данных:

```
shp:Рецензия a sh:NodeShape ;
    sh:targetClass :Рецензия ;
    sh:property [ sh:path :на_статью ; sh:class :Статья ;
                  sh:minCount 1 ; sh:maxCount 1 ;
                  sh:message "у рецензии должна быть ровно одна статья" ] ;
    sh:property [ sh:path :шкала ; sh:in (5 10) ; sh:message "шкала 5 или 10" ] ;
    sh:property [ sh:path :оценка ; sh:datatype xsd:decimal ;
                  sh:minInclusive 1 ; sh:maxInclusive 10 ;
                  sh:message "оценка от 1 до 10" ] .
```

Что делает форма. Цель: все рецензии. Первое ограничение: по свойству `на_статью` ровно одно значение, и это узел с типом Статья. Второе: значение `шкала` есть 5 или 10, любое другое число даёт нарушение. Третье: `оценка` есть десятичное число от 1 до 10 включительно. На чистых данных все четырнадцать рецензий проходят, и отчёт пуст.

На `review.owl` все три проходят: `Conforms: True`. Мелочь, на которой спотыкаются: оценка в файле записана как `7.0` с типом `xsd:decimal`. Форма с `sh:datatype xsd:integer` даст тринадцать нарушений по всем рецензиям с оценкой. Тип литерала в данных нужно знать точно.

9. Сравнение значений двух свойств

Ядро SHACL умеет сравнивать значения двух свойств одного узла: `sh:lessThan`, `sh:lessThanOrEquals`, `sh:equals`, `sh:disjoint`. Это то, чего в OWL нет вовсе.

«Дата назначения не позже даты сдачи»:

```
sh:property [ sh:path :назначено ; sh:lessThanOrEquals :сдано ;
              sh:message "назначено позже сдачи" ] .
```

Что делает форма. Для каждой рецензии берёт все значения `назначено` и все значения `сдано` и проверяет каждую пару: дата назначения не больше даты сдачи. У рецензии P8 даты сдачи нет, пар для сравнения нет,

нарушения нет. Чтобы потребовать само наличие даты сдачи, нужен `sh:minCount`.

На чистых данных проходит. Если перепутать стороны и написать `sh:path :сдано ; sh:lessThanOrEquals :назначено`, отчёт выдаст тринадцать нарушений: сравнение направлено от значения пути к значению второго свойства, и у каждой сданной рецензии дата сдачи больше даты назначения.

Требование 6.6 положения, «рецензия сдаётся в 21 день», нуждается в арифметике дат: срок равен дате назначения плюс 21 день. В ядре SHACL арифметики нет. Решение лекции: при загрузке данных в коде вычисляется свойство `срок`, а форма сравнивает две записанные даты:

```
sh:property [ sh:path :сдано ; sh:lessThanOrEquals :срок ;
              sh:message "6.6: рецензия сдана позже срока" ] .
```

Что делает форма. То же сравнение, где второе свойство `срок` записано заранее: дата назначения плюс 21 день. Для рецензии `рец_P5_Rv3` срок равен 24 сентября, сдана она 30 сентября, и отчёт называет её с сообщением про 6.6. Остальные двенадцать сданных рецензий уложились в срок.

Это разделение стоит запомнить: вычисление делается в коде, проверка записывается в данных. Второй путь, с арифметикой внутри запроса, показан в разделе 14.

10. Пути: не только одно свойство

`sh:path` не обязан быть одним свойством. Три вида составных путей встречаются чаще всего.

Обратный путь `[ sh:inversePath :на_статью ]`: от статьи к её рецензиям, хотя в данных стрелка направлена от рецензии к статье. Он нужен для требования 6.1, «у статьи не меньше двух сданных рецензий», потому что в файле записано только `на_статью`. Обратное свойство `имеет_рецензию` машина вывода лекции 2 выводила, а форма его не видит. Попытка новичка:

```
shp:Статья a sh:NodeShape ; sh:targetClass :Статья ;
    sh:property [ sh:path :имеет_рецензию ; sh:minCount 2 ; sh:message "6.1" ] .
```

Что делает форма. Для каждой статьи ищет значения свойства `имеет_рецензию` и требует хотя бы два. В файле такого свойства нет ни у одной статьи, значений ноль, и все восемь статей получают нарушение, включая P1 с двумя сданными рецензиями.

Отчёт: восемь нарушений, по одному на каждую статью, включая те, у которых две рецензии есть. Форма честно сообщает, что свойства `имеет_рецензию` в графе нет ни у кого. Правильная запись с обратным путём в разделе 11.

Последовательность `( :рецензент :из_организации )`: два шага подряд, от рецензии к организации рецензента. «У рецензента записана организация»:

```
sh:property [ sh:path ( :рецензент :из_организации ) ;
              sh:class :Организация ; sh:minCount 1 ;
              sh:message "у рецензента не записана организация" ] .
```

Что делает форма. Путь из двух шагов: от рецензии по `рецензент` к рецензенту, от него по `из_организации` к организации. В конце пути должно быть хотя бы одно значение, и оно должно быть узлом с типом Организация. Рецензия без рецензента и рецензент без организации дали бы нарушение у рецензии.

На чистых данных проходит.

Повторение `[ sh:zeroOrMorePath :зависит_от ]`: цепочка любой длины, например все положения, на которые опирается требование. Используется реже, потому что такие вопросы обычно отдают машине вывода или запросу.

11. Составные формы

Когда одного ограничения мало, формы соединяются.

Вложенная форма `sh:node`: значение свойства само должно соответствовать другой форме. «У каждого автора книги записано имя»:

```
shp:Книга a sh:NodeShape ; sh:targetClass :Книга ;
    sh:property [ sh:path :автор ; sh:node shp:Автор ] .
shp:Автор a sh:NodeShape ;
    sh:property [ sh:path :имя ; sh:minCount 1 ; sh:message "у автора нет имени" ] .
```

Что делает форма. Первая форма для каждой книги отправляет каждое значение `автор` на проверку второй форме. Вторая форма без цели, она применяется только к тем узлам, которые ей передали, и требует у узла хотя бы одно имя. Три автора без имени не проходят, и обе формы пишут об этом: вторая называет автора, первая называет книгу.

```
  Груббер   имя     у автора нет имени   Violation   MinCountConstraintComponent
  Лами      имя     у автора нет имени   Violation   MinCountConstraintComponent
  Поппер    имя     у автора нет имени   Violation   MinCountConstraintComponent
 к2     автор  Value does not conform to Shape shp:Автор  Violation  NodeConstraintComponent
 к2     автор  Value does not conform to Shape shp:Автор  Violation  NodeConstraintComponent
 к3     автор  Value does not conform to Shape shp:Автор  Violation  NodeConstraintComponent
```

Отчёт двухъярусный: внешняя форма сообщает, что значение не прошло вложенную, вложенная сообщает, что именно не так. У формы `shp:Автор` нет цели: она применяется только там, куда её позвали.

Квалифицированное число `sh:qualifiedValueShape` с `sh:qualifiedMinCount`: «не меньше N значений, удовлетворяющих форме». Так записывается 6.1, где считать нужно только сданные рецензии:

```
shp:Статья a sh:NodeShape ; sh:targetClass :Статья ;
    sh:property [
        sh:path [ sh:inversePath :на_статью ] ;
        sh:qualifiedValueShape [ sh:property [ sh:path :сдано ; sh:minCount 1 ] ] ;
        sh:qualifiedMinCount 2 ;
        sh:message "6.1: сданных рецензий меньше двух" ] .
```

Что делает форма. Цель: все статьи. Путь обратный: от статьи к рецензиям, у которых `на_статью` указывает на эту статью. Из найденных рецензий считаются только те, у которых есть `сдано`, это задаёт вложенная форма в `sh:qualifiedValueShape`. Таких должно быть не меньше двух. У P4 одна сданная рецензия, у P8 ни одной, обе получают нарушение. Несданные рецензии в счёт не идут.

```
  P4   6.1: сданных рецензий меньше двух   Violation
  P8   6.1: сданных рецензий меньше двух   Violation
```

У P4 одна рецензия, у P8 рецензия назначена и не сдана. Остальные шесть статей проходят. Путь в отчёте показан как пустой узел, потому что обратный путь не есть именованное свойство.

Логические связки `sh:and`, `sh:or`, `sh:not`, `sh:xone` соединяют формы. «Сданная рецензия имеет оценку» записывается как «либо есть оценка, либо рецензия не сдана»:

```
shp:Рецензия a sh:NodeShape ; sh:targetClass :Рецензия ;
    sh:or ( [ sh:property [ sh:path :оценка ; sh:minCount 1 ] ]
            [ sh:property [ sh:path :сдано ; sh:maxCount 0 ] ] ) ;
    sh:message "сданная рецензия без оценки" .
```

Что делает форма. Для каждой рецензии должна выполниться хотя бы одна из двух вложенных форм: есть оценка, или нет даты сдачи. Сданная рецензия без оценки не выполняет ни одной и даёт нарушение. Несданная рецензия P8 без оценки проходит по второй ветви.

На чистых данных проходит. Отрицание `sh:not [ sh:property [ sh:path :год ; sh:hasValue 1945 ] ]` на учебной библиотеке находит книгу к4.

12. Цели: к каким узлам применяется форма

До сих пор цель задавалась классом. Есть четыре способа.

Цель	Какие узлы проверяются	Когда удобно
`sh:targetClass :Книга`	все узлы с типом Книга и с типами его подклассов	обычный случай
`sh:targetNode :P3`	один названный узел	проверить конкретную запись
`sh:targetSubjectsOf :автор`	все узлы, у которых есть свойство автор	узлы без типа
`sh:targetObjectsOf :автор`	все значения свойства автор	проверить, кем бывают авторы

Про подклассы: цель по классу захватывает экземпляры подклассов по записанным в графе данных тройкам `rdfs:subClassOf`, без машины вывода. Если Учебник объявлен подклассом Книги, форма для книг проверит и учебники.

Цель по объектам свойства находит Груббера: «значение свойства автор должно быть автором», и отчёт называет его, хотя у Груббера нет типа и под цель по классу он не попал бы.

13. Закрытая форма

Обычно форма проверяет только то, о чём в ней сказано, лишние свойства ей безразличны. `sh:closed true` запрещает у узла любые свойства, кроме перечисленных в форме и в `sh:ignoredProperties`:

```
shp:Книга a sh:NodeShape ; sh:targetClass :Книга ;
    sh:closed true ; sh:ignoredProperties ( rdf:type ) ;
    sh:property [ sh:path :название ] ; sh:property [ sh:path :автор ] .
```

Что делает форма. Для каждой книги разрешены только свойства из формы, `название` и `автор`, и свойства из списка исключений, здесь `rdf:type`. Любое другое свойство у книги даёт нарушение, и отчёт называет это свойство. В учебном графе у всех четырёх книг есть `год`, которого в списке нет.

Отчёт: четыре нарушения, у каждой книги лишнее свойство `год`, потому что его нет в списке. Закрытая форма полезна для приёма данных по строгой схеме, где опечатка в имени свойства должна быть замечена.

14. Когда ядра не хватает: форма с запросом

Требование 6.2: рецензент не из организации авторов. Нужно сравнить организацию, полученную по пути рецензия → рецензент → организация, с организацией по пути рецензия → статья → организация авторов. Ядро SHACL сравнивает значения двух свойств одного узла, а здесь значения получены по двум разным путям. Ядра не хватает.

SHACL разрешает записать ограничение запросом SPARQL. Запрос возвращает узлы, которые нарушают требование:

```
<http://example.org/shapes> a owl:Ontology ;
    sh:declare [ sh:prefix "" ; sh:namespace "http://example.org/review.owl#"^^xsd:anyURI ] .

shp:Рецензия a sh:NodeShape ; sh:targetClass :Рецензия ;
    sh:sparql [
        sh:message "6.2: рецензент из организации авторов" ;
        sh:prefixes <http://example.org/shapes> ;
        sh:select """
            SELECT $this WHERE {
              $this :рецензент ?p .  ?p :из_организации ?o .
              $this :на_статью ?s .  ?s :организация_авторов ?o . }
        """ ] .
```

Что делает форма. Первые две строки объявляют префикс `:` для запроса. Форма нацелена на все рецензии. Запрос выполняется для каждой из них, `$this` подставляется вместо рецензии: находится её рецензент `?p`, его организация `?o`, её статья `?s`, и проверяется, что у статьи та же организация авторов `?o`. Если такие связи нашлись, рецензия попадает в отчёт. Для `рец_P3_Rv4` нашлись: рецензент Rv4 из ФИЦ ИУ, авторы P3 из ФИЦ ИУ.

```
  рец_P3_Rv4   6.2: рецензент из организации авторов   Violation
```

Переменная `$this` есть проверяемый узел. Четыре строки запроса те же четыре условия, что в правиле 6.2 из лекции 2, различается судьба результата: правило добавляет факт «нарушает», форма добавляет строку в отчёт. Блок `sh:declare` объявляет префиксы для запроса, без него внутри запроса пришлось бы писать полные адреса.

Тем же способом записывается 6.6 без предварительного вычисления срока: `FILTER (?сдано > ?назначено + "P21D"^^xsd:dayTimeDuration)`. Арифметика дат есть в SPARQL, в ядре SHACL её нет. Язык запросов целиком разбирается в лекции 4.

15. SHACL и OWL рядом

	OWL и машина вывода	SHACL
вопрос	что следует из записанного	всё ли записано и верно ли
мир	открытый: неизвестное не считается ложным	замкнутый: чего нет в графе, того нет
результат	новые факты, классификация, объяснение противоречия	отчёт о несоответствиях, граф не меняется
«у рецензии нет статьи»	статья есть, пока не названа	нарушение minCount
«рецензент не из организации авторов»	правило выводит факт «нарушает»	запрос даёт строку отчёта

Два следствия для практики. Первое: форма видит только записанное. Если нужная связь выводится машиной вывода (обратное свойство, транзитивная зависимость), то либо путь в форме переписывается (`sh:inversePath`), либо выведенные факты сначала материализуются, то есть записываются в граф, и только потом идёт проверка. Второе: параметр `inference` у pySHACL добавляет перед проверкой вывод по RDFS или OWL RL. В курсе он равен `"none"`: что проверяется, то и записано, и отчёт читается без сюрпризов.

Одно требование положения получает разные записи в зависимости от того, что нужно сделать с результатом: правило, когда нужен новый факт в графе, выражение класса, когда нужна классификация, форма, когда

нужен отчёт о данных, запрос, когда нужен список.

16. Типичные ошибки

- Форма на выведенное свойство: `имеет_рецензию` в `sh:path` без `sh:inversePath` и без материализации. Форма сообщает о нарушении у всех узлов.
- Не тот тип литерала: `xsd:integer` там, где в данных `xsd:decimal` или строка. Тип нужно смотреть в данных.
- Перепутанные стороны сравнения: `сдано lessThanOrEquals назначено` даёт нарушение у каждой сданной рецензии.
- Ограничение на содержание без ограничения на наличие: `sh:pattern` молчит у узлов, где свойства нет. Наличие требует `sh:minCount`.
- Сообщение без пункта требования: по отчёту нельзя понять, что именно нарушено.
- Ожидание, что предупреждение сохранит `Conforms: True`. Итог истинен только при пустом отчёте.
- Намеренная ошибка внесена в сдаваемый файл. Её место в копии данных.

17. Шпаргалка

Что нужно	Конструкция
объявить форму и её цель	`shp:X a sh:NodeShape ; sh:targetClass :Класс`
ограничение на свойство	`sh:property [ sh:path :свойство ; ... ]`
не меньше, не больше, ровно	`sh:minCount`, `sh:maxCount`
значение из класса, литерал типа, вид узла	`sh:class`, `sh:datatype`, `sh:nodeKind`
список, обязательное значение, диапазон, образец	`sh:in`, `sh:hasValue`, `sh:minInclusive`/`sh:maxInclusive`, `sh:pattern`
сравнить два свойства узла	`sh:lessThan`, `sh:lessThanOrEquals`, `sh:equals`, `sh:disjoint`
обратный путь, два шага, цепочка	`[ sh:inversePath :p ]`, `( :p :q )`, `[ sh:zeroOrMorePath :p ]`
вложенная форма, считать подходящие значения	`sh:node`, `sh:qualifiedValueShape` + `sh:qualifiedMinCount`
логика	`sh:and`, `sh:or`, `sh:not`, `sh:xone`
другие цели	`sh:targetNode`, `sh:targetSubjectsOf`, `sh:targetObjectsOf`
запретить лишние свойства	`sh:closed true ; sh:ignoredProperties (...)`
строгость, текст	`sh:severity sh:Warning`, `sh:message "..."`
ограничение запросом	`sh:sparql [ sh:select "SELECT $this WHERE {...}" ]`

18. Задачи для самопроверки

1. Написать форму «у каждой статьи записана организация авторов, и это организация». Ответ: `sh:targetClass :Статья`, `sh:property [ sh:path :организация_авторов ; sh:class :Организация ; sh:minCount 1 ]`.
2. Что выдаст форма с `sh:datatype xsd:integer` для оценки на `review.owl` и почему. Ответ: тринадцать нарушений, оценки записаны как `xsd:decimal`.
3. Почему форма для 6.1 через `:имеет_рецензию` находит нарушения у всех статей. Ответ: свойство выведено машиной вывода и в файле отсутствует, форма видит только записанное, нужен `sh:inversePath :на_статью`.

4. Данные содержат одно предупреждение и ни одного нарушения. Чему равен `Conforms`. Ответ: False, итог истинен только при пустом отчёте.
5. Записать требование «рецензия сдана не раньше назначения» и объяснить порядок сторон. Ответ: `sh:path :назначено ; sh:lessThanOrEquals :сдано`, сравнение идёт от значения пути к значению второго свойства.
6. Какое требование положения нельзя записать ядром SHACL и что делать. Ответ: 6.2, сравнение значений по двум разным путям, записывается через `sh:sparql`.